Lab 7
240968040

1. Write a program in CUDA to add two vectors using:

a) block size as N

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <cuda_runtime.h>

__global__ void vectorAdd(int *A, int *B, int *C, int N) {
    int i = threadIdx.x;
    if (i < N) {
        C[i] = A[i] + B[i];
    }
}

int main() {
    int N = 100;
    int size = N * sizeof(float);

    int *h_A = (int *)malloc(size);
    int *h_B = (int *)malloc(size);
    int *h_C = (int *)malloc(size);

    srand(time(NULL));
    for (int i = 0; i < N; i++) {
        h_A[i] = (int)(rand() % 10);
        h_B[i] = (int)(rand() % 10);
    }

    int *d_A, *d_B, *d_C;
    cudaMalloc(&d_A, size);
    cudaMalloc(&d_B, size);
    cudaMalloc(&d_C, size);

    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

    vectorAdd<<<1, N>>>(d_A, d_B, d_C, N);

    cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);

    for (int i = 0; i < 100; i++) {
        printf("C[%d] = %d + %d = %d\n", i, h_A[i], h_B[i], h_C[i]);
    }
```

```
    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);
    free(h_A);
    free(h_B);
    free(h_C);

    return 0;
}
```

Output:

```
pcap1@PC-36:~/Documents/240968040/week 7$ nvcc q1a.cu -o q1a
pcap1@PC-36:~/Documents/240968040/week 7$ ./q1a
C[0] = 0 + 7 = 7
C[1] = 5 + 3 = 8
C[2] = 2 + 6 = 8
C[3] = 4 + 0 = 4
C[4] = 8 + 5 = 13
C[5] = 7 + 4 = 11
C[6] = 5 + 5 = 10
C[7] = 3 + 8 = 11
C[8] = 1 + 0 = 1
C[9] = 2 + 4 = 6
C[10] = 8 + 9 = 17
C[11] = 6 + 3 = 9
C[12] = 9 + 9 = 18
C[13] = 6 + 0 = 6
C[14] = 2 + 1 = 3
C[15] = 1 + 2 = 3
C[16] = 8 + 6 = 14
```

b) N threads
Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <cuda_runtime.h>

__global__ void vectorAdd(int *A, int *B, int *C, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) {
        C[i] = A[i] + B[i];
    }
}

int main() {
```

```
    int N = 500;
    int size = N * sizeof(int);

    int *h_A = (int *)malloc(size);
    int *h_B = (int *)malloc(size);
    int *h_C = (int *)malloc(size);

    srand(time(NULL));
    for (int i = 0; i < N; i++) {
        h_A[i] = (int)(rand() % 10);
        h_B[i] = (int)(rand() % 10);
    }

    int *d_A, *d_B, *d_C;
    cudaMalloc(&d_A, size);
    cudaMalloc(&d_B, size);
    cudaMalloc(&d_C, size);

    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

    int threadsPerBlock = 32;
    int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock; // 4 blocks = 128 threads total

    vectorAdd<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, N);

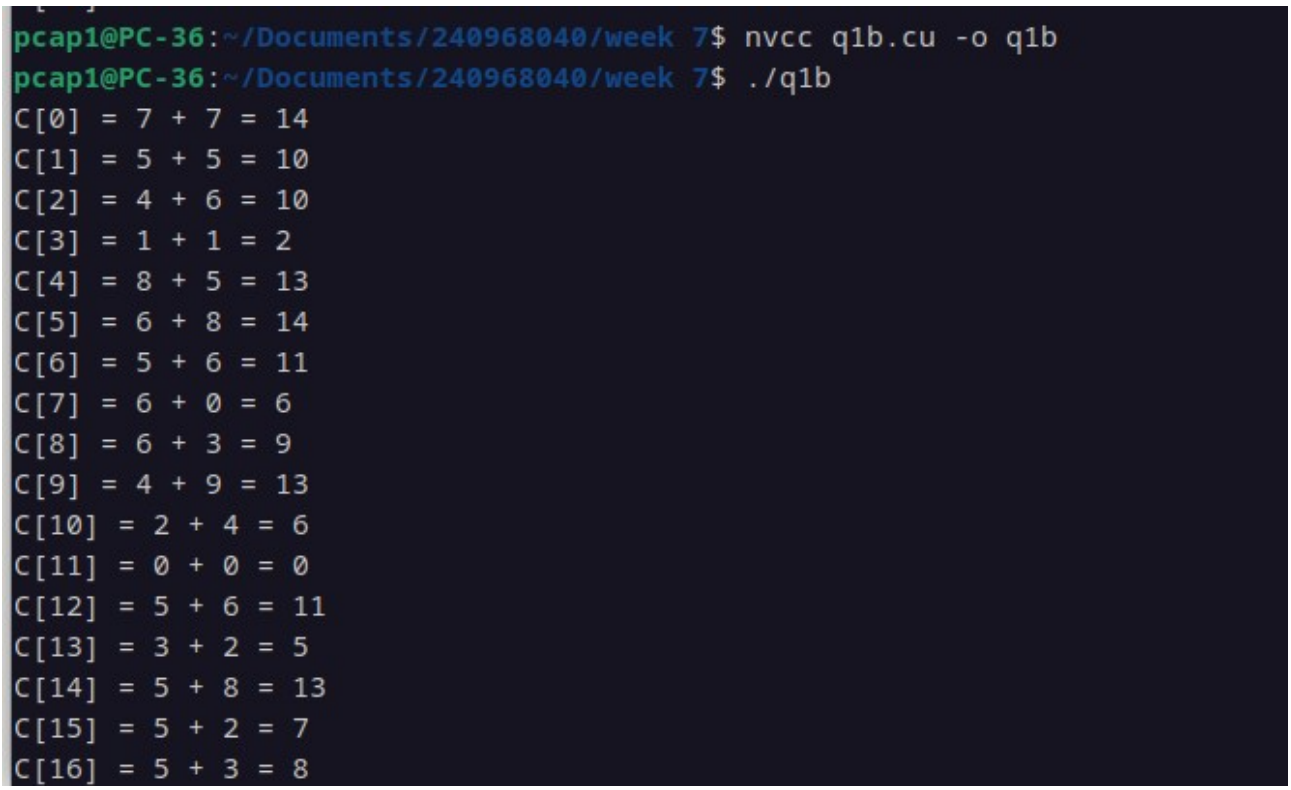
    cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);

    for (int i = 0; i < 250; i++) {
        printf("C[%d] = %d + %d = %d\n", i, h_A[i], h_B[i], h_C[i]);
    }

    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);
    free(h_A);
    free(h_B);
    free(h_C);

    return 0;
}
```

Output



c) Varying block size to handle N elements with thread size constant as 256

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <cuda_runtime.h>

__global__ void vectorAdd(float *A, float *B, float *C, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) {
        C[i] = A[i] + B[i];
    }
}

int main() {
    int N = 1024;
    int size = N * sizeof(float);

    float *h_A = (float *)malloc(size);
    float *h_B = (float *)malloc(size);
    float *h_C = (float *)malloc(size);

    srand(time(NULL));
    for (int i = 0; i < N; i++) {
        h_A[i] = (float)(rand() % 10);
        h_B[i] = (float)(rand() % 10);
    }
```

```
    float *d_A, *d_B, *d_C;
    cudaMalloc(&d_A, size);
    cudaMalloc(&d_B, size);
    cudaMalloc(&d_C, size);

    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

    int blockSize = 256;
    int gridSize = (N + blockSize - 1) / blockSize;

    vectorAdd<<<gridSize, blockSize>>>(d_A, d_B, d_C, N);

    cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);

    for (int i = 0; i < 500; i++) {
        printf("C[%d] = %.1f + %.1f = %.1f\n", i, h_A[i], h_B[i], h_C[i]);
    }

    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);
    free(h_A);
    free(h_B);
    free(h_C);

    return 0;
}
```

Output

```
pcap1@PC-36:~/Documents/240968040/week 7$ nvcc q1c.cu -o q1c
pcap1@PC-36:~/Documents/240968040/week 7$ ./q1c
C[0] = 6.0 + 2.0 = 8.0
C[1] = 8.0 + 1.0 = 9.0
C[2] = 2.0 + 8.0 = 10.0
C[3] = 5.0 + 8.0 = 13.0
C[4] = 0.0 + 1.0 = 1.0
C[5] = 2.0 + 3.0 = 5.0
C[6] = 0.0 + 4.0 = 4.0
C[7] = 8.0 + 6.0 = 14.0
C[8] = 7.0 + 2.0 = 9.0
C[9] = 5.0 + 7.0 = 12.0
C[10] = 3.0 + 5.0 = 8.0
C[11] = 6.0 + 6.0 = 12.0
C[12] = 0.0 + 4.0 = 4.0
C[13] = 2.0 + 3.0 = 5.0
C[14] = 4.0 + 5.0 = 9.0
C[15] = 6.0 + 1.0 = 7.0
C[16] = 9.0 + 6.0 = 15.0
C[17] = 2.0 + 1.0 = 3.0
```

q

3) Write a program in CUDA to process a 1D array containing angles in radians to generate sine of the angles in the output array. Use appropriate function.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <cuda_runtime.h>

// Kernel to calculate the sine of each angle in radians
__global__ void computeSine(float *d_angles, float *d_sines, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) {
        d_sines[i] = sinf(d_angles[i]);
    }
}

int main() {
    int N = 100;
    int size = N * sizeof(float);

    float *h_angles = (float *)malloc(size);
    float *h_sines = (float *)malloc(size);

    srand(time(NULL));
    for (int i = 0; i < N; i++) {
        h_angles[i] = ((float)rand() / (float)RAND_MAX) * 6.28318530718f;
```

```
    }

    // Allocate device memory
    float *d_angles, *d_sines;
    cudaMalloc(&d_angles, size);
    cudaMalloc(&d_sines, size);

    cudaMemcpy(d_angles, h_angles, size, cudaMemcpyHostToDevice);

    int blockSize = 256;
    int gridSize = (N + blockSize - 1) / blockSize;

    computeSine<<<gridSize, blockSize>>>(d_angles, d_sines, N);

    cudaMemcpy(h_sines, d_sines, size, cudaMemcpyDeviceToHost);

    for (int i = 0; i < 100; i++) {
        printf("sin(%.2f radians) = %.4f\n", h_angles[i], h_sines[i]);
    }

    cudaFree(d_angles);
    cudaFree(d_sines);
    free(h_angles);
    free(h_sines);

    return 0;
}
```

Output:

```
pcap1@PC-36:~/Documents/240968040/week 7$ nvcc q2.cu -o q2
pcap1@PC-36:~/Documents/240968040/week 7$ ./q2
sin(2.53 radians) = 0.5708
sin(0.10 radians) = 0.1017
sin(6.25 radians) = -0.0286
sin(3.95 radians) = -0.7250
sin(0.95 radians) = 0.8153
sin(2.72 radians) = 0.4071
sin(4.35 radians) = -0.9336
sin(0.43 radians) = 0.4136
sin(1.36 radians) = 0.9779
sin(2.10 radians) = 0.8615
sin(2.56 radians) = 0.5454
sin(2.35 radians) = 0.7147
sin(0.28 radians) = 0.2779
sin(3.72 radians) = -0.5502
```